

Novas Abordagens Arquiteturais de APIs: RESTful, GraphQL, WebSockets, WebHooks, HTTP Streaming

Objetivo da Aula

Conhecer as principais abordagens arquiteturais para APIs e aplicá-las em projetos NodeJS.

Apresentação

Na área de tecnologia, as ferramentas estão em constante evolução. Cabe a nós, desenvolvedores, compreender as novas tendências e selecionar quais componentes podem ser adotados em cada contexto. Vamos analisar algumas tendências arquiteturais para a construção de APIs, observando semelhanças e diferenças, além do melhor contexto para a utilização de cada uma.

Para a gerência de dados, veremos como se comportam os modelos REST e GraphQL. Através de um exemplo prático, utilizaremos GraphQL em projeto NodeJS, observando a mudança na forma de utilização frente ao REST.

Veremos como Web Sockets e HTTP Streaming permitem oferecer uma manutenção de estados que não é natural do protocolo HTTP, através de fluxos contínuos de informação. Também iremos analisar os Webhooks, um recurso de interoperabilidade muito utilizado para serviços na Web, permitindo sincronizar as ações entre dois sistemas distintos de forma bastante simples.

1. RESTful

Os Wb Services do tipo **RESTful**, que implementam a arquitetura **REST**, se tornaram cada vez mais comuns, principalmente com o advento das plataformas móveis. Sua comunicação com dados no formato **JSON** e perfeita integração com o protocolo HTTP, permitem a qualquer cliente tratar o conjunto de serviços oferecidos como simples requisições de recursos no HTTP.

Como são obedecidos os princípios da arquitetura REST, temos uma relação direta entre as **rotas**, os métodos do protocolo **HTTP** e as ações de um **CRUD** agindo em segundo plano. O quadro seguinte demonstra essa relação para uma entidade Produto.

Quadro 1: Relação entre métodos HTTP, endpoints e operações do CRUD.

Método HTTP	Endpoint para a Rota	Operação do CRUD
POST	/produtos	CREATE
GET	/produtos	READ
	/produtos/:id	
PUT	/produtos/:id	UPDATE
DELETE	/produtos/:id	DELETE

Fonte: Elaboração própria.

Embora não tenha estado (**Stateless**), por executar sobre o protocolo HTTP, um Web Service RESTful pode usar o sistema de **cache**, diminuindo o tempo de resposta para operações que geram o mesmo resultado. Uma boa opção para implementação de cache de entidades é o **Redis**, que trabalha em memória, o que representa uma grande melhoria de desempenho quando comparado com o acesso ao banco de dados.

Em termos de segurança, um Web Service RESTful pode usar a autenticação no próprio protocolo HTTP, ou trabalhar com o sistema de **tokens**. Em geral, o modelo baseado em tokens é mais utilizado, permitindo, inclusive, autenticação por terceiros, com base em **OAuth** e **OIDC**.

Por se tratar de uma ferramenta voltada para a **interoperabilidade**, já que será usada sempre por terceiros, sendo acionada a partir de chamadas de outros sistemas, uma boa **documentação** é essencial. Nesse ponto, o NodeJS permite o uso de **Swagger**, que segue o padrão **Open API**, gerando a documentação de forma padronizada e consistente. O uso do Web Service por terceiros traz outras preocupações relevantes, como a necessidade de versionamento de serviços e dados, planejamento para a desativação da API e manutenção de versões paralelas, para que não haja quebra de funcionalidades nos clientes.



Destaque

A tecnologia GraphQL busca diminuir a dependência da interface que ocorre no REST, diminuindo também a necessidade de versionamento.

Utilizando uma estratégia consistente para implantação e manutenção, os Web Services RESTful representam o meio mais simples para a comunicação entre plataformas heterogêneas. Com isto, estão se tornando cada vez mais populares no mercado de desenvolvimento.

2. GraphQL

A tecnologia **GraphQL** (Graph Query Language) foi desenvolvida pela equipe do Facebook (Meta) como uma alternativa ao modelo REST. Ela tem a vantagem de poder retornar múltiplos formatos de resposta para um mesmo **endpoint**, pois define uma sintaxe de pesquisa simplificada.

Quando utilizamos GraphQL, enviamos para o endpoint quais campos serão retornados a partir do **schema** utilizado no servidor, permitindo a obtenção exata dos dados desejados, ao invés de retornar uma entidade inteira. Como temos o envio de uma consulta, não há necessidade de controle de versão, pois se um campo é adicionado para pesquisas novas, ele não irá impactar as antigas.

Como podemos observar na listagem seguinte, a palavra **type** permite definir um **schema** para consulta.

```
type Aluno {
  id: ID
  nome: String
}
```

Definida a estrutura, as perguntas são feitas no **endpoint** com o comando **query**, como no exemplo seguinte, onde é selecionado o aluno com id **123**, e retornado o campo **nome**, no formato **JSON**.

```
query getAlunoById {  
  aluno(id: "123") {  
    nome  
  }  
}
```

A alteração de dados ocorre com o uso de **mutation**.

```
mutation createUser(name: "Ana") {  
  id  
  name  
}
```

Ao trabalhar com **GraphQL** no **NodeJS**, podemos utilizar o **Apollo Server**, que é voltado especificamente para GraphQL, ou utilizar um servidor **Express** e a biblioteca de ligação **graphql-http**. Crie um diretório vazio, abra no VS Code e digite os comandos da listagem seguinte, para configurar o projeto.

```
npm init -y  
npm install express graphql-http
```

Agora, crie o schema, no arquivo **produto-schema.js** com o código seguinte.

```
const {GraphQLObjectType, GraphQLString, GraphQLSchema} =  
  require('graphql');  
const produtos = [{id: '1', nome: 'Teclado'},  
  {id: '2', nome: 'Mouse'}, {id: '3', nome: 'Monitor'}]  
const ProdutoType = new GraphQLObjectType({  
  name: 'Produto',  
  fields: () => ({id: {type: GraphQLString}},
```

```

        nome: {type: GraphQLString}})
    })
    const ProdutoQuery = new GraphQLObjectType({
      name: 'findProduto',
      fields: {produto: {
        type: ProdutoType, args:{id:{type:GraphQLString}},
        resolve(parent, args) {
          return produtos.find((prod)=>prod.id==args.id)
        }
      }}
    })
    module.exports = new GraphQLSchema({query: ProdutoQuery})

```

Em **ProdutoType** temos a definição dos campos no atributo **fields**, enquanto **ProdutoQuery** define um **argumento** de busca com o nome **id**, e utiliza **resolve** para retornar o produto com o id informado. Ao final, definimos o **schema**, com o fornecimento da **query**.

Já podemos criar o servidor, no arquivo **index.js**, com o código da listagem seguinte. Note que a rota **/graphql** aceita todos os métodos HTTP e direciona o tratamento para **createHandler**.

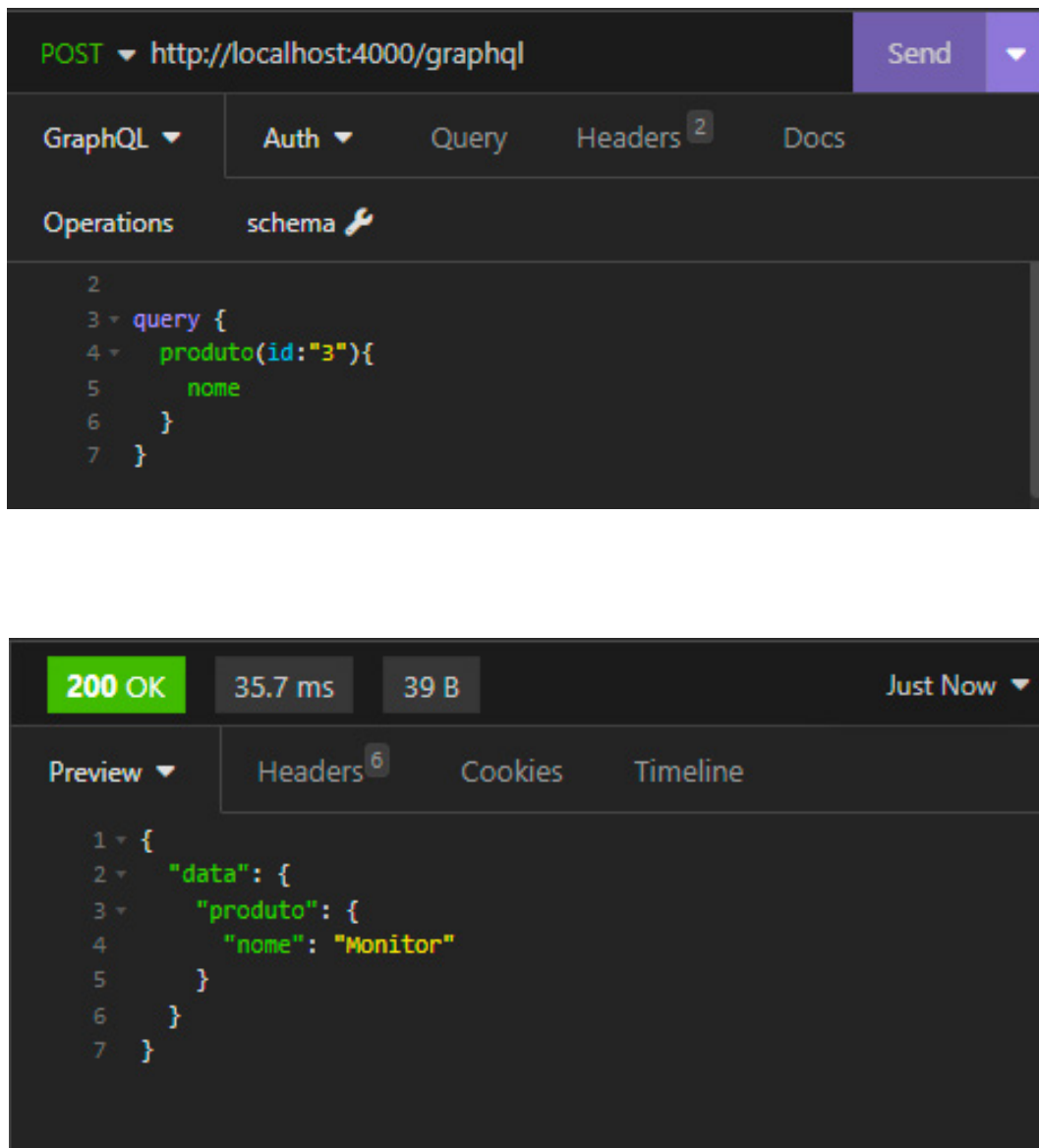
```

const express = require('express');
const {createHandler} =
  require('graphql-http/lib/use/express');
const schema = require('./produto-schema');
const app = express();
app.all('/graphql', createHandler({ schema }));
app.listen({ port: 4000 });
console.log('Listening to port 4000');

```

O servidor será executado com o comando **node index** e passará a escutar a porta **4000**. Para testar, você pode utilizar o **Insomnia**.

Figura 1: Uso de Insomnia para testes com GraphQL - Captura de tela do Insomnia



Fonte: Elaboração própria.

3. Web Sockets

Através de **Web Sockets**, temos uma comunicação bidirecional sobre socket TCP/IP, sendo comum sua utilização em jogos, chats e aplicativos de tempo real, pois fornece um estilo de **manutenção de estado** inviável para o modelo de requisição e resposta do HTTP. Você pode identificar a utilização do protocolo pelos prefixos **ws** ou **wss**.

A tecnologia foi padronizada pela **W3C**, apresentando plena compatibilidade com **HTML 5**. Hoje, temos muitos jogos que executam diretamente nas páginas, tendo como base o Canvas e a tecnologia de Web Sockets.

Para utilizar Web Sockets no NodeJS, precisamos da biblioteca **ws**. Crie um diretório vazio, abra no VS Code e execute os comandos seguintes.

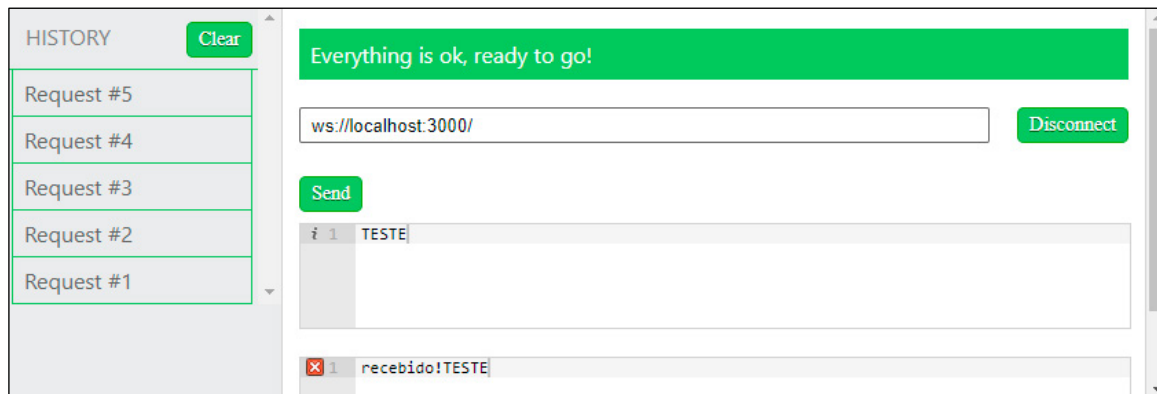
```
npm init -y  
npm install ws express
```

Em seguida, crie o arquivo **index.js**, utilizando o código da listagem seguinte.

```
const WebSocket = require('ws');  
const express = require('express');  
const app=express()  
const onMessage = (ws, d) => {ws.send(`recebido! ${d}`)};  
const onConnection = (ws, req) => {  
    ws.on('message', data => onMessage(ws, data));  
    console.log(`onConnection`);  
}  
const server = app.listen(3000,()=>console.log('executando'));  
const wss = new WebSocket.Server({server});  
wss.on('connection', onConnection);
```

Observe como o **WebSocket** encapsula o servidor **Express** e direciona a conexão para a função **onConnection**. Com a execução da função, o evento **message** é associado a uma callback, onde os dados recebidos são retornados para o cliente, via **send**, precedidos da palavra **recebido**. Executando o aplicativo com o comando **node index** e acessando a partir de uma ferramenta de teste, como o plugin **Smart Websocket Client**, para o Chrome, é possível enviar a informação e observar o retorno.

Figura 2: Plugin Smart Websocket Client – Captura de tela do Chrome



Fonte: Elaboração própria.

4. Webhooks

Como o próprio nome diz, um **Webhook** representa um “gancho” na Web, onde um site, ao ter uma determinada rota acionada, pode enviar uma solicitação para outro site. Em geral, pode se tornar uma boa ferramenta para garantia de **interoperabilidade** entre dois sistemas que executam no protocolo HTTP.

O principal objetivo de um Webhook é garantir a comunicação entre duas APIs. Eles também são chamados de **APIs reversas** ou **APIs de push**, sendo oferecido por muitos sites, como o GitHub.

Para testar os Webhooks, crie um diretório vazio, abra no VS Code e utilize os comandos da listagem seguinte.

```
npm init -y
npm i express node-webhooks
```

Em seguida, crie o arquivo **rotas.js** e utilize o código da listagem seguinte.

```
const express = require("express")
const webhooks = require('node-webhooks')
const routes = express.Router();
const dados = [1,2,3];
routes.get("/", (req, res) => { res.json({valores: dados}); })
```



```

routes.post("/", (req, res) => {
  dados.push(req.body.valor);
  const hooks = registerHooks();
  hooks.trigger('callback_hook',
    {msg: "novo valor", data: req.body.valor});
  res.json({valores: dados});
})
const registerHooks = () => {
  return new webhooks({
    db: {'callback_hook':
      ['http://localhost:8005/webhook-client']}
  });
}
module.exports = routes

```

Vamos começar a análise pela função **registerHooks**, onde é instanciado um objeto **webhooks**, tendo como parâmetro o conjunto de endereços que será acionado de forma automática. A resposta para a rota raiz, via método **POST**, irá adicionar o novo valor no repositório, que no caso é apenas um vetor, registrar os hooks, disparar para os endereços definidos, com um objeto JSON na nova requisição, e retornar o vetor completo para o cliente.

Agora, vamos criar o arquivo **index.js** e utilizar o código da listagem seguinte.

```

const express=require('express');
const rotas=require('./rotas');
const app1=express(), app2=express();
app1.use(express.json()); app2.use(express.json());
app1.use("/app1", rotas);
app1.listen(3000, () => console.log('app1'));
app2.post('/webhook-client', async(req, res) => {
  console.log('Inside Callback hook', req.body);

```

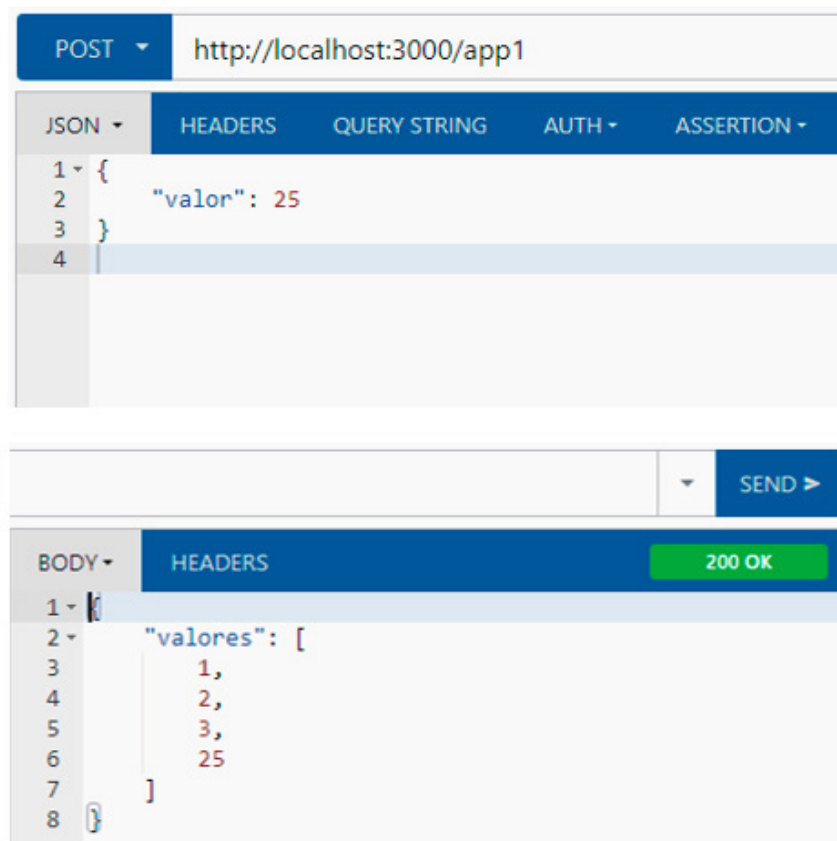
```
return res.status(200).end();

});

app2.listen(8005, () => console.log('app2'));
```

Foram definidos dois servidores Express no mesmo arquivo, sendo que **app1** utiliza as **rotas** definidas anteriormente e responde na porta **3000**, enquanto **app2** é o serviço ativado pelo **hook**, atendendo na porta **8005**. A resposta do segundo servidor é bem simples, imprimindo os dados e retornando status 200. Inicie os servidores com o comando **node index** e depois efetue o **POST** para **http://localhost:3000/app1**, passando o atributo numérico **valor** no corpo da requisição, através de alguma ferramenta de teste, como o Boomerang.

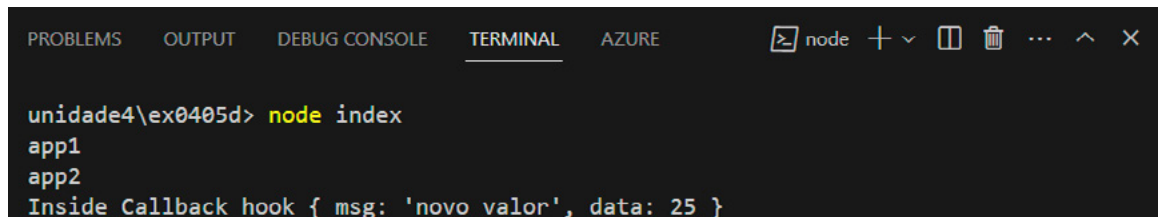
Figura 3: Chamada POST em app1 com Boomerang - Captura de tela do Chrome



Fonte: Elaboração própria.

Ao mesmo tempo em que a chamada inclui um valor no repositório, **app2** irá imprimir no terminal o novo valor.

Figura 4: Chamada de app2 acionada pelo Web Hook – captura de tela do VS Code)



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  AZURE  [node icon] node + - [icon] [icon] ... ^ X

unidade4\ex0405d> node index
app1
app2
Inside Callback hook { msg: 'novo valor', data: 25 }
```

Fonte: Elaboração própria.

5. HTTP Streaming

A tecnologia de **HTTP Streaming** mantém uma **conexão aberta** para **fluxo** de dados contínuo, como no fornecimento de vídeo e conteúdos grandes. Dessa forma, existe uma manutenção de estados que não ocorreria em uma requisição HTTP comum, já que temos envio de dados sob demanda. Do lado do cliente, deve ser adotado um padrão **Observer**, onde um evento de recepção de dados é acionado sempre que ocorre a chegada de um novo bloco de informações, direcionando para a função de callback associada.

Para demonstrar a utilização de Http Streaming no NodeJS, vamos utilizar o framework **Koa**, uma opção ao uso de Express. Após criar um diretório vazio e abrir no VS Code, utilize os comandos apresentados a seguir.

```

npm init -y
npm install koa
```

No modelo do Koa, temos uma callback para a rota com utilização de um objeto **Context**, fornecendo acesso a diversos objetos do ambiente, ao invés de controlar apenas a requisição e a resposta, tendo até mesmo uma variável de estado, que simplifica o gerenciamento de sessões. Ele foi desenvolvido pelo mesmo time do Express, com o objetivo de criar uma versão menor, mais robusta e com maior suporte a novas tecnologias Web. Crie o arquivo **index.js**, utilizando o código seguinte.

```

const http = require('http');
const Koa = require('koa');
const fs = require('fs');
const app = new Koa();
```

```
app.use(async (ctx) => {
  if (ctx.request.url === '/measurements.json') {
    ctx.response.set('content-type', 'application/json');
    ctx.body = fs.createReadStream('./measurements.json',
                                  {highWaterMark: 16});
  });
http.createServer(app.callback()).listen(3000);
```

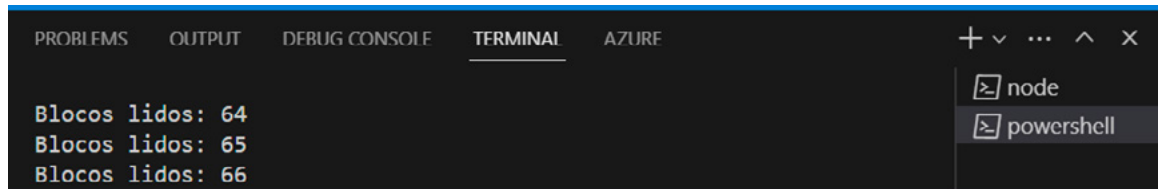
Como podemos observar, tudo que temos é a verificação da URL a partir do atributo **request** do contexto, e sendo o endereço correto, definimos o conteúdo da resposta no atributo **request** e associamos o atributo **body** a uma **stream** de leitura de arquivo. Nesse ponto, temos o **Http Streaming** implementado, mas é necessário criar um arquivo com o nome **measurements.json**, contendo grande quantidade de linhas no formato JSON, localizado na raiz do projeto. Crie o arquivo **cliente.js**, utilizando o código seguinte, para efetuar um teste.

```
fetch('http://localhost:3000/measurements.json')
  .then(async (response) => {
    const reader = response.body.getReader();
    let readResult, i = 0;
    do{
      readResult = await reader.read();
      if(readResult.value)
        console.log(`Blocos lidos: ${++i}`);
    }while(!readResult.done)
  });
```

Nosso cliente utiliza apenas o **fetch** padrão do Java Script, e nele capturamos o **stream** do corpo da resposta, através de **getReader**. Em seguida, os blocos são lidos, à medida em que são enviados pelo servidor, até que não haja mais dados e o atributo **done** de **readResult** se torne verdadeiro.

Abra dois terminais, no primeiro utilize **node index**, ativando o servidor, e no segundo **node cliente**. Observe a saída obtida, demonstrando a recepção de dados segmentada.

Figura 5: Execução do cliente para HTTP Streaming – captura de tela do VS Code)



Fonte: Elaboração própria.

Considerações Finais da Aula

Aqui, vimos as características da arquitetura REST e como o GraphQL traz maior flexibilidade ao uso de endpoints, com a obtenção de dados baseada em pesquisas, e não em formatos específicos. Ficou claro que o versionamento pode ser flexibilizado no GraphQL, já que o acréscimo de atributos não impacta nas consultas pré-existentes.

Em termos de fluxo contínuo, analisamos os Web Sockets, muito utilizado em jogos, chats e aplicativos de tempo real, e o HTTP Streaming, que normalmente é usado para transmissão de vídeo e áudio. Vimos a implementação de ambos os modelos em projetos NodeJS, além de conhecer o Smart WebSocket Client, um plug-in do Chrome que facilita muito o teste de Web Sockets.

Também vimos a implementação de um Web Hook na prática, construindo dois serviços REST, onde o primeiro aciona o segundo de forma automática, a partir de uma requisição específica. Qualquer outro serviço, mesmo em outras arquiteturas, poderia ser acionado pela mesma técnica. Não menos importante, conhecemos o Koa, uma alternativa mais flexível e robusta que o Express para a implementação de serviços na Web.

Materiais Complementares



GraphQL — como testar utilizando Insomnia

2021, Gustavo Lima.

O artigo demonstra a utilização do Insomnia para acessar servidores GraphQL, o que permite definir uma boa estratégia para especificação de testes com esse tipo de arquitetura.

Link para acesso: <https://medium.com/revista-dtar/graphql-como-testar-utilizando-insomnia-db993a65844c> (acesso em: 14 set. 2023.)



Koa – Next Generation Web Framework for NodeJs

Koa.

O site oficial do Koa oferece um pequeno conjunto de informações, mas o suficiente para saber como instalar e utilizar o framework, incluindo alguns exemplos de implementação com diferentes arquiteturas Web.

Link para acesso: <https://koajs.com/> (acesso em: 14 set. 2023.)



Montando API GraphQL em Node.js com Apollo Server e MongoDB.

2020, Emerson Pereira.

O artigo mostra o passo-a-passo para integrar GraphQL e MongoDB via NodeJS.

Link para acesso: https://medium.com/@emerson_pereira/montando-api-graphql-em-node-js-com-apollo-e-mongodb-6b5089ff10d3 (acesso em: 14 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. ISBN 9788565837484. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidade-Paginas%3d1%26codigoRegistro%3d92164%2392164&i=8>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. ISBN 9788536533100. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidade-Paginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. ISBN 9786558110217. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 14 set. 2023.